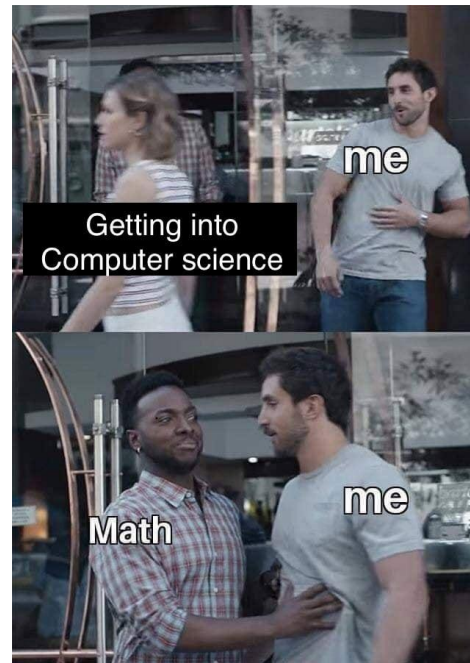


D1: Asymptotic Notations, Proof Methods, and the Potential Method



Follow along with the
discussion worksheet!



Sec 19: Th 4:30-5:30pm 2153 GGBL
IA: Anthony Chalhoub

Agenda

- ▶ Introduction
- ▶ Let's talk about EECS 376!
- ▶ Asymptotic Notations
- ▶ Proof Methods Review
- ▶ The Potential Method

Introduction



Hello!!

- ▶ Anthony Chalhoub
- ▶ Senior - Computer Science
 - ▶ Engineering Global Leadership; Honors
 - ▶ International & Ross Business minors
- ▶ Second time teaching EECS 376 🖱️
- ▶ Office hours
 - ▶ Mon 12:30-3:30 PM Leinweber 2421
 - ▶ Wed 12:30-2:30 PM Leinweber 2421
- ▶ Email: anthch@umich.edu



Discussion Attendance Policy

- ▶ You may attend any section (though it is nice to be consistent)
- ▶ If you attend at least 10 of 14 discussions throughout the semester, then you get 3% for free for your overall grade and your exam scores weigh 3% less

Assignment	Weight
Homeworks	40%
Exams (midterm and final)	56-60%
Discussion attendance	0-3%
Course evaluations	0-1%
Total	100%

Let's talk about
EECS 376!



An Overview of EECS 376

- ▶ **Is this a math class?** We are borrowing this nice tool called *proof* from math to explain ideas in CS
- ▶ **How much math do I need?** Little to none. We don't expect you to do two-pages-long algebra or calculus in this class. It's more about the formality of math language
- ▶ **Why is this class called *Foundations of CS*?** Don't get deceived! *Foundation* $\neq$ *introductory*! It means we are dealing with the *foundational problems* in the discourse of CS (e.g., what problems can/ can't computer solve?)

Is EECS 376 part 2 of EECS 203?

We DO need stuff from 203, but the two courses are fundamentally different

- ▶ **Throughout the course**
 - ▶ Proof methods- will see today
 - ▶ Asymptotic notation- will see today
 - ▶ Logical expressions
 - ▶ Set theory
 - ▶ Graph theory
- ▶ **Part 1: Design and Analysis of Algorithm**
 - ▶ Recursion + Master Theorem
- ▶ **Part 2: Computability**
 - ▶ Countability and Diagonalization
 - ▶ Pigeonhole Principle

Is EECS 376 part 2 of EECS 203?

We DO need stuff from 203, but the two courses are fundamentally different

- ▶ **Part 3: Complexity**
 - ▶ Function (mapping)
- ▶ **Part 4: Randomness in Algorithms**
 - ▶ Counting
 - ▶ Discrete Probabilities
 - ▶ Expectation
- ▶ **Part 5: Cryptography**
 - ▶ Modular Arithmetic
 - ▶ Divisibility and Modular Inverses

The Fight Against Imposter Syndrome

- ▶ The beginning of a new semester is extremely overwhelming
 - ▶ You're entering a class that has a reputation for being difficult
 - ▶ There are a million other factors in your life like work, family, health, etc.
- ▶ Always remember that you can do it!
- ▶ You belong in this class and in the EECS program
- ▶ Every member of the course staff believes you can succeed and it's our job to help make that happen
 - ▶ Please take advantage of all the resources in this class - office hours, Piazza, lecture notes, discussions, discussion worksheets
 - ▶ If/when you feel lost, come to office hours and talk to us! A lot of this material will not make sense the first time you see it, but it's very rewarding to understand so let us help you!!

Asymptotic Notations



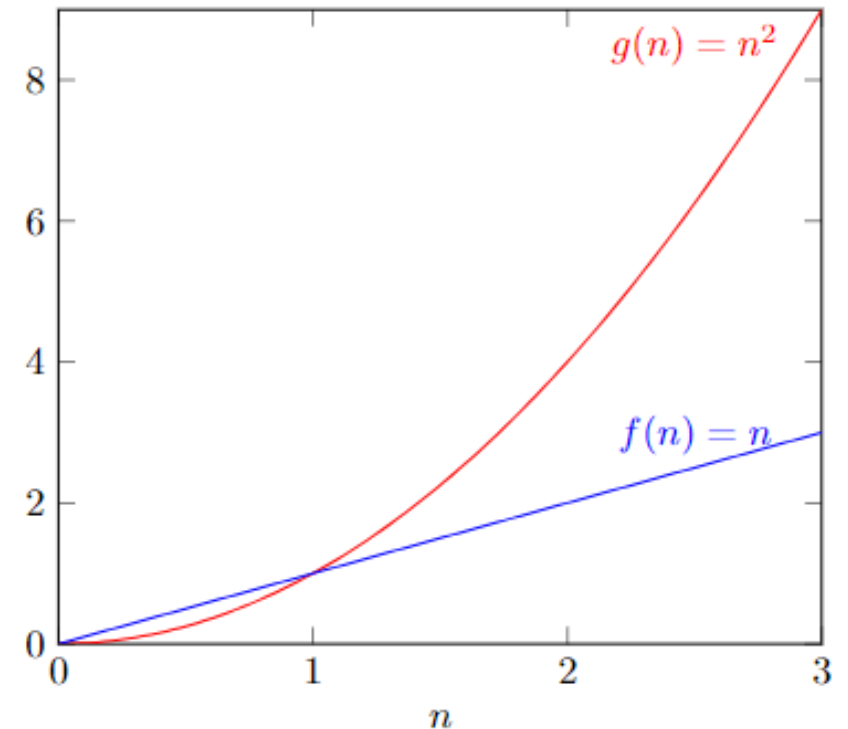
Big-O Bounds a Function from Above

Let $f(n)$ and $g(n)$ be positive functions:

- ▶ If there exist positive constants M and n_0 such that for all $n \geq n_0$, we have $f(n) \leq M \cdot g(n)$, then we say

$$f(n) = O(g(n))$$

- ▶ For example, on the right we could pick $M = 1$ and $n_0 = 1$ to show $n = O(n^2)$

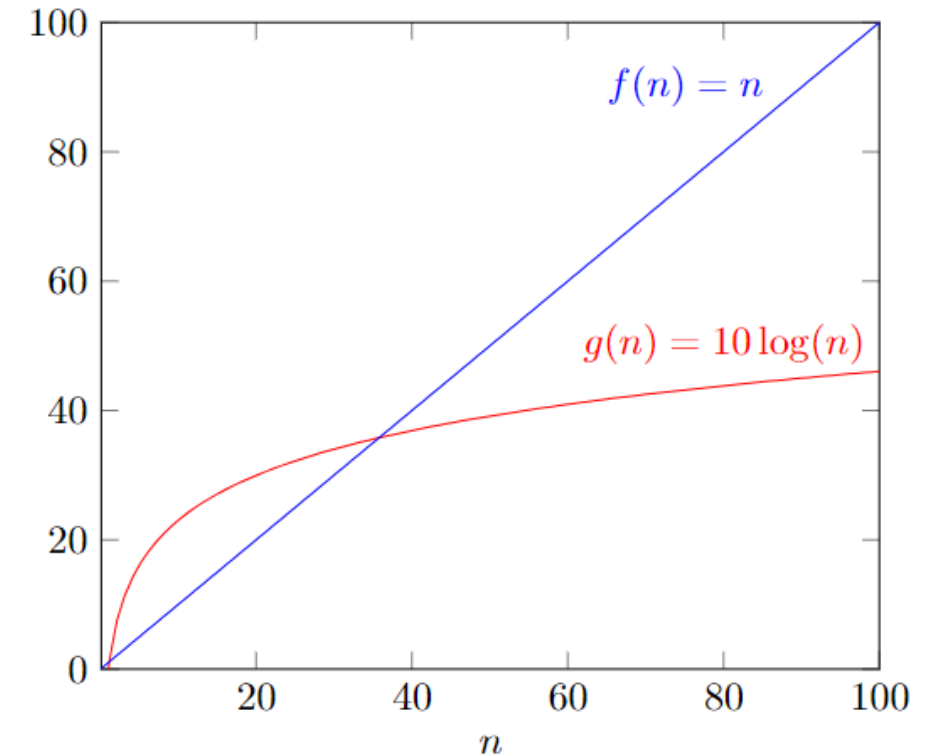


Big- Ω Bounds a Function from Below

Let $f(n)$ and $g(n)$ be positive functions:

- If there exist positive constants M and n_0 such that for all $n \geq n_0$, we have $f(n) \geq M \cdot g(n)$, then we say

$$f(n) = \Omega(g(n))$$



Big- Θ Bounds a Function from Both Directions

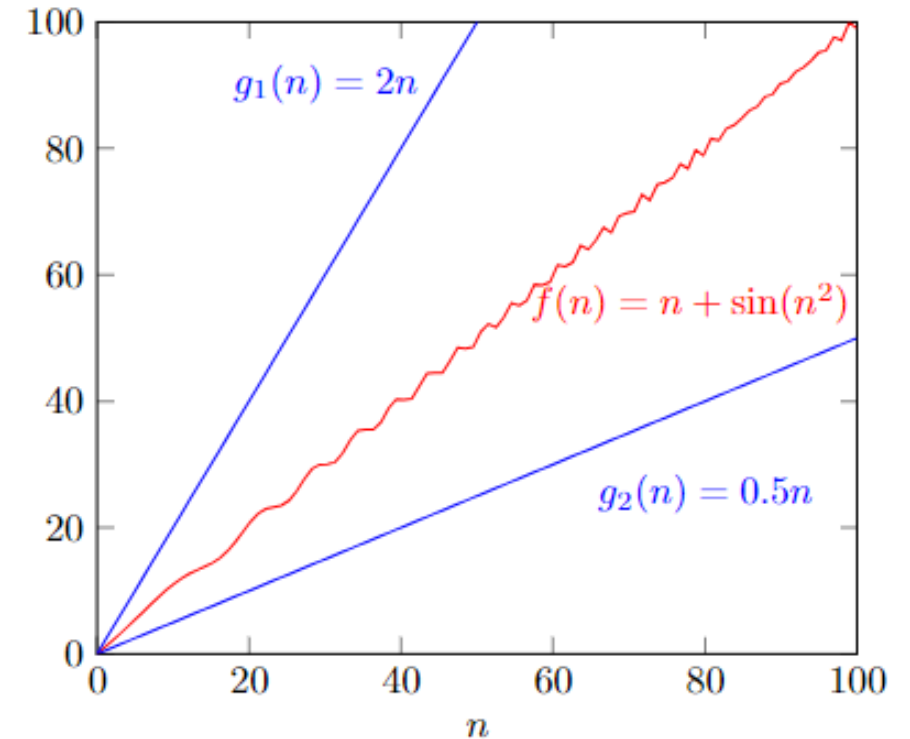
Let $f(n)$ and $g(n)$ be positive functions:

- ▶ If $f(n) = O(g(n))$ **AND** $f(n) = \Omega(g(n))$, then we say

$$f(n) = \Theta(g(n))$$

- ▶ For example, if we want to prove $n + \sin(n^2) = O(n)$, set $f(n) = n + \sin(n^2)$ and $g(n) = n$, we pick $n_0 = 0$ and

- ▶ $M = 2$ to show $\forall n \geq n_0, f(n) \leq M \cdot g(n) \Rightarrow f(n) = O(g(n))$
- ▶ $M = 0.5$ to show $\forall n \geq n_0, f(n) \geq M \cdot g(n) \Rightarrow f(n) = \Omega(g(n))$



Key Takeaway

- ▶ Big-O is only a **bound from above**; big- Ω is only a **bound from below**
- ▶ Key properties of big- O (similarly for big- Ω and big- Θ)
 - ▶ "positive constant M ... $f(n) \leq M \cdot g(n)$ ": $f(n)$ is upper bounded by some **constant multiple** of $g(n)$
 - ▶ "positive constant n_0 ... for all $n \geq n_0$ ": the upper bound holds for all n above **some constant "threshold"**
- ▶ Read [Handout 1](#)!!!!

Some Information Theory Stuff

Discuss: What is the size of inputs to the following algorithms?

- Algorithm X takes in an array of k ASCII characters and print them one by one
- Algorithm Y takes in an integer k and print “EECS 376” k times

- ▶ **Why different?** When we deal with arrays, for simplicity we usually implicitly assume that all elements are (or bounded by) the same size (since they are the same data type), making the **total size proportional to the number of elements**
 - ▶ Example: ASCII characters are represented as 8-bit bytes with the most significant bit set to 0- so the assumption is true
- ▶ But this assumption **may not be true in general!** One example we deal with is big integers
 - ▶ $\text{int } x = 5$ (binary: 101) takes $\lceil \log_2 x \rceil = 3$ bits
 - ▶ $\text{int } y = 50$ (binary: 110010) takes $\lceil \log_2 y \rceil = 6$ bits
 - ▶ $\text{int } z = 50000$ (binary 1100 0011 0101 0000) takes $\lceil \log_2 z \rceil = 16$ bits

Proof Methods Review



Definition: An integer is **even** if and only if $x = 2k$ for some integer k .

Proof by Contrapositive ($\neg Q \Rightarrow \neg P$)

- ▶ When a direct proof ($P \Rightarrow Q$) seems difficult: it is **equivalent**, and sometimes **easier**, to prove the *contrapositive* (If not Q then not P)
- ▶ **Example:** Let $f(x) = x + 2$. Using proof by contrapositive, prove that $f(x)$ is even if and only if x is even.
 - ▶ Direction 1: If x is even, then $f(x)$ is even.
 - ▶ Direction 2: If $f(x)$ is even, then x is even.

Poll: What is the contrapositive of Direction 2?

- A. If x is odd, then $f(x)$ is odd.
- B. If $f(x)$ is odd, then x is odd.

Proof by Contradiction

- ▶ Assume $\neg P$ (P is false)
- ▶ Create some false/ illogical statements, for example:
 - ▶ $1 = 2$
 - ▶ a is odd (when we assumed it to be even)
 - ▶ G is acyclic (when we assumed it is not acyclic)
- ▶ Because we can arrive at some illogical conclusion with P being false, P cannot be false. Must be true.

An example of proof by contradiction in EECS 376

- ▶ Statement: “Problem P **is not** solvable by algorithm A”
- ▶ Proof by contradiction:
 - ▶ Suppose problem P **is** solvable by algorithm A
 - ▶ [something happens]
 - ▶ which means [some truth breaks down, e.g., Pigeonhole Principle]
 - ▶ 376 pigeonholes, none of the 377 pigeons share the same pigeonhole
 - ▶ Contradiction. Therefore, P must be true.

SPOILER ALERT

We'll see similar idea in
Unit 2: Computability

Definition: A binary string is any sequence of zero or more binary digits (0 or 1).

Proof By Induction

- ▶ Used to prove that a statement $P(n)$ is true for all positive/non-neg integers n .
- ▶ Two things to prove: **base case**, and **inductive case**
- ▶ **Example:** Suppose I want to compute $x!$. Consider the following algorithm:

```
FACTORIAL( $x$ ):  
    if  $x = 0$  or  $x = 1$  then  
        return 1 // Base case  
    return  $x \cdot \text{FACTORIAL}(x - 1)$ 
```

SPOILER ALERT

We'll see this soon in Unit 1:
Algorithms to prove the
correctness of recursion!

- ▶ **Application of induction:**
 - ▶ **Base case:** $0! = 1! = 1$ - correct
 - ▶ **Inductive step:** Assuming the **recursive call** $\text{FACTORIAL}(k - 1)$ **is correct** for some k , prove that $\text{FACTORIAL}(k)$ is correct.

Upsilon Time!

- ▶ Who is Upsilon?
 - ▶ Professor Upsilon (Υ) is a brilliant but occasionally absent-minded computer scientist, known for their groundbreaking insights into algorithms—and the occasional “oopsie” moments
- ▶ Today, Υ is here to present you with some (questionable) proofs to help you review proof methods
- ▶ Discuss what's wrong with their proofs!



Upsilon Proofs of The Day

1

Claim: There is no even primes greater than 2.

Proof: 3 is a prime that is greater than 2 and it is not even, so there must be no even primes greater than 2.

2

Claim: If n^2 is even, then n is even.

Proof: If n is even, then $n = 2k$ for some integer k . It follows that

$$n^2 = 4k^2 = 2 \cdot 2k^2$$

3

Claim: 37 is the largest prime smaller than 41.

Proof: 37 is prime since its only divisors are 1 and 37.



Upsilon Proof 1

Claim: There is no even primes greater than 2.

Proof: 3 is a prime that is greater than 2 and it is not even, so there must be no even primes greater than 2.

Problem: Attempt to prove a for all (negation of there exists) statement with a counterexample, what about other primes greater than 2?

Upsilon Proof 2

Claim: If n^2 is even, then n is even.

Proof: If n is even, then $n = 2k$ for some integer k . It follows that

$$n^2 = 4k^2 = 2 \cdot 2k^2$$

Problem: Wrong direction!

Upsilon Proof 3

Claim: 37 is the largest prime smaller than 41.

Proof: 37 is prime since its only divisors are 1 and 37.

Problem: Incomplete! Why is 37 the *largest* prime smaller than 41? Do we have other prime that is in between 37 and 41?

Potential Method



Potential Method

- ▶ The **potential method** is a useful technique to reason about the amount of time an algorithm takes before **halting**
- ▶ **Definition:** A function $s: A \rightarrow \mathbb{R}$ that maps the "state" of the algorithm (i.e., its internal variables) to a (finite) real number is a valid potential function if:
 - ▶ s **strictly decreases** by at least some **fixed positive constant** $C > 0$ with **every "unit of work"**
 - ▶ s has some **finite lower bound** B , i.e., the algorithm is guaranteed to terminate once $s \leq B$
- ▶ The first condition ensures that $s \leq B$ **will eventually occur** within a finite number of work units
- ▶ The second condition ensures that the algorithm **will terminate** within the number of works units

Upsilon Time!

- Consider the following algorithm:

```
x ← user's input
while x > 0 do
  if x is even then
    x ← x + 1
  else
    x ← x - 3
```

- Upsilon claims that there is no potential functions (in terms of x) that is strictly decreasing because x increases when the user's input is even. Therefore, the program never halt
- Is Upsilon's claim true? Why or why not?

Definition: A function $s: A \rightarrow \mathbb{R}$ that maps the "state" of the algorithm (i.e., its internal variables) to a (finite) real number is a valid potential function if:

- s **strictly decreases** by at least some **fixed positive constant** $C > 0$ with **every "unit of work"**
- s has some **finite lower bound** B , i.e., the algorithm is guaranteed to terminate once $s \leq B$

Main Takeaway

- ▶ YOU define the "unit of work"
- ▶ Make sure **BOTH** conditions are satisfied when defining a potential function
 - ▶ s **strictly decreases** by at least some **fixed positive constant** $C > 0$ with **every "unit of work"**
 - ▶ s has some **finite lower bound** B , i.e., the algorithm is guaranteed to terminate once $s \leq B$

Your turn!

- **Task:** Define and analyze a potential function (in terms of x and/or y) for your algorithm to prove that it halts on all inputs
- **At-Home Exercise:** Mix-and-match the blocks and try to create potential functions

```
x ← user's input
y ← user's input
while x > 0 and y > 0 do
    z ← user's input
    [if-block]
    [else-block]
```

Solutions

if-block A

if z is even then

$x \leftarrow x - 1$

$y \leftarrow y + 1$

if-block B

if x is odd then

$x \leftarrow x + 1$

if-block C

if $z < 5$ then

$x \leftarrow x - 2$

$y \leftarrow y + 1$

else-block A

else

$y \leftarrow y - 1$

else-block B

else

$x \leftarrow x + 1$

$y \leftarrow y - 1$

else-block C

else

$x \leftarrow x - 2$

Attendance!



Sign in here :)

Questions? I'll stick around for a bit

